Name-Janhvi Singh_EnrollmentNo.-GGV/24/05129

```
//1.Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.


import java.io.*; import
java.util.*;
class Farmer {
    String name, fruit, city;     int quantity;
    Farmer(String n, String f, String c, int q) {         name = n; fruit = f;
city = c; quantity = q;     }
    public String toString() {
        return String.format("%-15s %-10s %-15s %-10d", name, fruit, city,
quantity);
    }
}
public class FarmerSystem {
    public static void main(String[] args) throws IOException {
        Farmer[] farmers = {
            new Farmer("Ramesh", "Mango", "Nagpur", 500),
new Farmer("Suresh", "Apple", "Shimla", 1200),
   new Farmer("Mahesh", "Papaya", "Pune", 300),             new
Farmer("Ganesh", "Mango", "Nashik", 800),             new Farmer("Dinesh",
"Apple", "Kashmir", 1500),             new Farmer("Somesh", "Litchi", "Bihar",
400),             new Farmer("Raj", "Papaya", "Surat", 200),             new
Farmer("Amit", "Mango", "Ratnagiri", 1100),             new Farmer("Vijay",
"Apple", "Manali", 950),             new Farmer("Anil", "Litchi", "Dehradun",
600)         };         saveToFile("before_sort.txt", farmers);
        Arrays.sort(farmers, (a, b) -> a.quantity - b.quantity);         saveToFile("after_sort.txt",
farmers);         System.out.println("Data sorted and saved to notepad files.");
        // Search Operation
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Farmer Name to search: ");
        String searchName = sc.next();
        boolean found = false;         for(Farmer f :
farmers) {
            if(f.name.equalsIgnoreCase(searchName)) {
                System.out.println("Found: " + f);
                found = true;                 break;
}
}
if(!found) System.out.println("Farmer not found!");     }
static void saveToFile(String filename, Farmer[] data) throws IOException {
PrintWriter pw = new PrintWriter(new FileWriter(filename));
pw.println(String.format("%-15s %-10s %-15s %-10s", "Name", "Fruit", "City", "Quantity"));
pw.println("------------------------------------------------------------");
for(Farmer f : data) pw.println(f);
pw.close();
}
}
//Sample User Input & Output:


Enter Farmer Name to search: Ramesh
```

```
Found: Ramesh
Mango      Nagpur
500

//2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.


Perform search for a given fruit with given quality.
import java.util.*;
public class FruitManager {
    public static void main(String[] args) {         Scanner sc = new
Scanner(System.in);
        String[] fruits = {"Mango", "Apple", "Papaya"};
        int[] quantities = new int[3];         String[] qualities =
new String[3];
        for(int i=0; i<3; i++) {
            System.out.print("Enter quantity for " + fruits[i] + ": ");
            int q = sc.nextInt();
            if(q > 1000) q = q / 2; // Store as half            quantities[i] = q;
            System.out.print("Enter quality (Good/Bad/Average) for " + fruits[i] +
": ");
            qualities[i] = sc.next();         }
        System.out.print("\nSearch - Enter Fruit Name: ");
        String sFruit = sc.next();
        System.out.print("Enter Quality: ");
        String sQual = sc.next();
        boolean found = false;         for(int i=0; i<3;
i++) {
            if(fruits[i].equalsIgnoreCase(sFruit) &&
qualities[i].equalsIgnoreCase(sQual)) {
                System.out.println("Match found! Quantity: " + quantities[i]);
                found = true;
            }
        }
        if(!found) System.out.println("No match found.");
    }
}


//Sample User Input & Output:


Enter quantity for Mango: 1200
Enter quality for Mango: Good ...
Search - Enter Fruit Name: Mango
Enter Quality: Good
Match found! Quantity: 600

//3.There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.


The available orchards in a tree-view.
Search for a given fruit in the available orchards after taking input from the user & print the tree.
import java.util.*;
class Orchard {
```

```
    String id, fruit, quality;
    Orchard(String id, String f, String q) { this.id = id; this.fruit = f;
this.quality = q; } }
public class OrchardTree {
    public static void main(String[] args) {
        List<Orchard> list = new ArrayList<>();         list.add(new Orchard("A",
"Apple", "Good"));         list.add(new Orchard("B", "Litchi", "Good"));
list.add(new Orchard("C", "Mango", "Good"));         list.add(new
Orchard("D", "Apple", "Bad"));
        System.out.println("--- Tree View of Orchards ---");
        System.out.println("Root (Farm)");
        for(Orchard o : list) {
            System.out.println(" |-- Orchard " + o.id + " [" + o.fruit + " (" + o.quality + ")]");
}
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter fruit to search: ");
        String search = sc.next();
        System.out.println("Search Results:");
        for(Orchard o : list) {
            if(o.fruit.equalsIgnoreCase(search)) {
                System.out.println("Found in Orchard " + o.id + " with " + o.quality + " quality.");
            }
        }
    }
}

//Sample User Input & Output: --- Tree View of Orchards ---


Root (Farm)
 |-- Orchard A [Apple (Good)] ...
Enter fruit to search: Apple Found in Orchard A
with Good quality.
Found in Orchard D with Bad quality.


//4Take a map provided as a graph & display all the subtrees of connected points.

import java.util.*;
public class GraphSubtrees {
    static Map<Integer, List<Integer>> adj = new HashMap<>();     static Set<Integer>
visited = new HashSet<>();
    public static void main(String[] args) {
        // Sample graph connections         addEdge(1, 2);
addEdge(2, 3);
        addEdge(4, 5);         addEdge(6, 6); //
Isolated
        System.out.println("Connected Subtrees in the Map:");
        for(int node : adj.keySet()) {             if(!visited.contains(node)) {
                System.out.print("Subtree: ");
                bfs(node);
                System.out.println();
            }
        }
    }
    static void addEdge(int u, int v) {         adj.putIfAbsent(u, new
ArrayList<>());         adj.putIfAbsent(v, new ArrayList<>());
        adj.get(u).add(v);
```

```
adj.get(v).add(u);     }
    static void bfs(int start) {
        Queue<Integer> q = new LinkedList<>();
        q.add(start);         visited.add(start);
while(!q.isEmpty()) {             int node =
q.poll();
            System.out.print(node + " ");             for(int neighbor :
adj.get(node)) {                 if(!visited.contains(neighbor)) {
visited.add(neighbor);
                    q.add(neighbor);
                }
            }
        }
    }
}
//Sample User Input & Output:


Connected Subtrees in the Map:
Subtree: 1 2 3
Subtree: 4 5
Subtree: 6


//5.Enter entry point & goal point & produce the matching subtree of the graph.


import java.util.*;
public class PathSubtree {
    static Map<String, List<String>> graph = new HashMap<>();
    public static void main(String[] args) {         graph.put("A",
Arrays.asList("B", "C"));         graph.put("B", Arrays.asList("D", "E"));
graph.put("C", Arrays.asList("F"));         graph.put("D", new
ArrayList<>());         graph.put("E", new ArrayList<>());
graph.put("F", new ArrayList<>());
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Entry Point (A-F): ");
        String start = sc.next().toUpperCase();
        System.out.println("Matching Subtree starting from " + start + ":");
        displaySubtree(start, 0);     }
    static void displaySubtree(String node, int level) {         for(int i=0; i<level; i++)
System.out.print("  ");
        System.out.println("-> " + node);         if(graph.containsKey(node)) {
            for(String child : graph.get(node)) {                 displaySubtree(child, level + 1);
            }
        }
    }
}

//Sample User Input & Output:


Enter Entry Point: A Matching Subtree
starting from A: -> A
  -> B
    -> D
    -> E
  -> C
```

```
    -> F


//6.Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

import java.util.*;
public class ShortestPath {     static
class Edge {         int target,
weight;
        Edge(int t, int w) { target = t; weight = w; }
    }
    public static void main(String[] args) {
        int nodes = 5;
        List<List<Edge>> adj = new ArrayList<>();
        for(int i=0; i<nodes; i++) adj.add(new ArrayList<>());
        adj.get(0).add(new Edge(1, 10));         adj.get(0).add(new Edge(2, 3));
adj.get(1).add(new Edge(2, 1));         adj.get(1).add(new Edge(3, 2));
adj.get(2).add(new Edge(1, 4));         adj.get(2).add(new Edge(3, 8));
adj.get(2).add(new Edge(4, 2));         adj.get(3).add(new Edge(4, 7));
adj.get(4).add(new Edge(3, 9));
        int[] dist = new int[nodes];
        Arrays.fill(dist, Integer.MAX_VALUE);         dist[0] = 0;
        PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.comparingInt(i  -> dist[i]));
pq.add(0);
        while(!pq.isEmpty()) {             int u =
pq.poll();             for(Edge e : adj.get(u)) {
                if(dist[u] + e.weight < dist[e.target]) {                    dist[e.target] = dist[u]
+ e.weight;
                    pq.add(e.target);
                }
            }
        }
        System.out.println("Shortest distances from Node 0:");
        for(int i=0; i<nodes; i++) {
            System.out.println("To Node " + i + " distance is " + dist[i]);
        }
    }
}

//Sample User Input & Output:


Shortest distances from Node 0:
To Node 0 distance is 0
To Node 1 distance is 7
To Node 2 distance is 3
To Node 3 distance is 9
To Node 4 distance is 5


//7. Write a program to demonstrate the use of all keywords made for exception handling.

public class ExceptionDemo {
    static void checkAge(int age) throws ArithmeticException {
        if(age < 18) {
            throw new ArithmeticException("Access denied - Under 18");
        } else {
            System.out.println("Access granted!");
```

```
        }
    }
    public static void main(String[] args) {
        try {
            System.out.println("Testing 'try' block...");
            checkAge(15);
        } catch (ArithmeticException e) {
            System.out.println("Caught 'throw' using 'catch': " + e.getMessage());
        } finally {
            System.out.println("The 'finally' block always executes.");
        }
        System.out.println("End of program.");
    }
}
//Sample User Input & Output:


Testing 'try' block...
Caught 'throw' using 'catch': Access denied - Under 18 The 'finally' block always
executes.
End of program.

//8.Write a program to withraw money from a bank account, if the withdrawn money is greater than the
balance, then it throws NoMoneyException.


class NoMoneyException extends Exception {     public
NoMoneyException(String message) {
        super(message);
    }
}
class BankAccount {     double balance;
BankAccount(double b) { balance = b; }
    void withdraw(double amount) throws NoMoneyException {
        if(amount > balance) {
            throw new NoMoneyException("Insufficient Balance! Your balance is: " +
balance);
}
        balance -= amount;
        System.out.println("Withdrawal successful! Remaining: " + balance);
    }
}
public class BankSystem {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(5000);
        java.util.Scanner sc = new java.util.Scanner(System.in);
        System.out.print("Enter amount to withdraw: ");         double amt =
sc.nextDouble();
        try {
            account.withdraw(amt);
        } catch (NoMoneyException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}


//Sample User Input & Output:
```

```


Enter amount to withdraw: 6000
Error: Insufficient Balance! Your balance is: 5000.0

//9. Write a program which enables to use a platform by a train one at a time


class Platform {
    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is entering the platform...");
        try {
            Thread.sleep(2000); // Train stays for 2 seconds
        } catch (Exception e) {}
        System.out.println(trainName + " has left the platform.");
    }
}
class Train extends Thread {
    Platform p;
    String name;
    Train(Platform p, String n) { this.p = p; this.name = n; }
    public void run() {
        p.usePlatform(name);
    }
}
public class RailwaySystem {
    public static void main(String[] args) {
        Platform mainPlatform = new Platform();
        Train t1 = new Train(mainPlatform, "Rajdhani Express");
        Train t2 = new Train(mainPlatform, "Shatabdi Express");
        Train t3 = new Train(mainPlatform, "Duronto Express");
        t1.start();         t2.start();
t3.start();
    }
}
//Sample User Input & Output:


Rajdhani Express is entering the platform...
Rajdhani Express has left the platform.
Shatabdi Express is entering the platform...
...

//10.Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff
deliver orders if he is free.


class Restaurant {
    boolean kitchenFree = true;
    synchronized void placeOrder(String order) throws InterruptedException {
        while (!kitchenFree) {
            System.out.println("Waiter: Kitchen is busy. Waiting...");
            wait();
        }
        System.out.println("Waiter: Order placed for " + order);
        kitchenFree = false;
        notify();     }
    synchronized void deliverOrder() throws InterruptedException {
```

```
        while (kitchenFree) {
            System.out.println("Chef: No orders to cook. Waiting...");
            wait();
        }
        System.out.println("Chef: Delivering the order!");
        kitchenFree = true;
        notify();
    }
}
public class KitchenProcess {
    public static void main(String[] args) {         Restaurant res = new
Restaurant();
        new Thread(() -> {
            try {
                res.placeOrder("Pasta");                 res.placeOrder("Pizza");
            } catch (Exception e) {}
        }).start();
        new Thread(() -> {
            try {
                while(true) {                     res.deliverOrder();
Thread.sleep(3000);
                }
            } catch (Exception e) {}
        }).start();
    }
}
//Sample User Input & Output:


Waiter: Order placed for Pasta Chef:
Delivering the order!
Waiter: Order placed for Pizza Chef:
Delivering the order!
```